



Université : École Nationale Supérieure d'Ingénieurs de
Tunis

Département : Génie Électrique

Rapport du Mini-Projet

Détection et Analyse des Composantes du Visage

Étudiantes :

Hammami Nour
Arfaoui Eya
Kaabi Maram

Encadrant :

Charfi Maher

Année universitaire : 2025–2026

Table des matières

1	Introduction	2
2	Environnement, outils utilisés et installation	3
2.1	Python et installation	3
2.2	Création d'un environnement virtuel	3
2.3	Librairies utilisées	4
2.3.1	OpenCV	4
2.3.2	os	4
2.3.3	datetime	4
2.3.4	Caméra USB	4
2.3.5	Carte graphique NVIDIA	5
3	Code source complet	7
4	Jeu de données et captures	9
4.1	Captures d'écran	9
5	Conclusion	14

Chapitre 1

Introduction

Ce rapport présente un système complet permettant la détection du visage, des yeux et du sourire en temps réel à l'aide de la bibliothèque OpenCV.

Il inclut :

- l'installation de l'environnement Python,
- les outils logiciels et matériels utilisés,
- le code détaillé du programme,
- un jeu de captures d'écran illustrant les résultats.

Chapitre 2

Environnement, outils utilisés et installation

Ce chapitre présente les outils logiciels et matériels nécessaires au fonctionnement du projet.

2.1 Python et installation



FIGURE 2.1 – *
Logo Python 3

Python est utilisé comme langage principal pour ce projet en raison de :

- sa simplicité et rapidité de développement,
- la disponibilité d'OpenCV,
- sa compatibilité avec les systèmes Linux/Windows.

Installation sous Linux

```
sudo apt update  
sudo apt install python3 python3-pip
```

Installation sous Windows

Téléchargement depuis : <https://www.python.org>
Cocher l'option : Add Python to PATH.

2.2 Création d'un environnement virtuel

```
python3 -m venv venv
```


Activation :

- Linux : `source venv/bin/activate`
- Windows : `venv\Scripts\activate`

2.3 Librairies utilisées

2.3.1 OpenCV



FIGURE 2.2 – *
Logo OpenCV

OpenCV est indispensable pour :

- la capture vidéo,
 - la détection faciale,
 - la gestion de la caméra,
 - l’affichage en temps réel.
- Installation :

```
pip install opencv-python
```

2.3.2 os

Utilisé pour :

- créer automatiquement le dossier des captures,
- vérifier l’existence de fichiers.

2.3.3 datetime

Utilisé pour générer des horodatages uniques pour nommer chaque capture.

2.3.4 Caméra USB

La caméra joue un rôle essentiel dans le fonctionnement de ce projet, car elle fournit en continu un flux vidéo utilisé pour analyser chaque image en temps réel. Elle permet de tester la robustesse du système de détection faciale dans différentes conditions (luminosité, distance, orientation du visage, etc.).

- capturer un flux vidéo en temps réel,
- permettre l’analyse image par image (frame-by-frame),
- vérifier le bon fonctionnement de l’algorithme en situation réelle,
- évaluer la qualité de détection selon l’environnement.



FIGURE 2.3 – Caméra USB utilisée pour la capture vidéo

2.3.5 Carte graphique NVIDIA

Même si le script actuel fonctionne sur CPU, une carte graphique NVIDIA est un élément essentiel dans les projets de vision par ordinateur modernes.

Elle permet :

- l'accélération du traitement d'images,
- l'exécution de modèles IA lourds,
- l'entraînement rapide de réseaux neuronaux,
- l'optimisation des calculs via CUDA.

Pourquoi NVIDIA ?

Les cartes NVIDIA sont les seules à supporter **CUDA**, une technologie permettant d'exécuter des calculs massivement parallèles.

Elles sont indispensables pour :

- YOLO, CNN, RetinaFace,
- reconstructions 3D,
- traitement temps réel haute résolution.

Image de la carte NVIDIA



FIGURE 2.4 – Carte graphique NVIDIA utilisée (emplacement réservé)

Chapitre 3

Code source complet

```
1 import cv2
2 import os
3 from datetime import datetime
4
5 # 1) Pr paration du dossier
6 output_dir = "captures"
7 if not os.path.exists(output_dir):
8     os.makedirs(output_dir)
9
10 # 2) Chargement des classifieurs Haar
11 face_cascade_path = "haarcascade_frontalface_default.xml"
12 eye_cascade_path = "haarcascade_eye.xml"
13 smile_cascade_path = "haarcascade_smile.xml"
14
15 face_cascade = cv2.CascadeClassifier(face_cascade_path)
16 eye_cascade = cv2.CascadeClassifier(eye_cascade_path)
17 smile_cascade = cv2.CascadeClassifier(smile_cascade_path)
18
19 # 3) Cam ra
20 cap = cv2.VideoCapture(0)
21
22 # 4) Boucle principale
23 while True:
24     ret, frame = cap.read()
25     if not ret:
26         break
27
28     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
29
30     faces = face_cascade.detectMultiScale(gray, 1.3, 5)
31
32     for (x, y, w, h) in faces:
33         cv2.rectangle(frame, (x,y), (x+w, y+h), (0,255,255), 2)
34         roi = gray[y:y+h, x:x+w]
35         roi_c = frame[y:y+h, x:x+w]
36
37         eyes = eye_cascade.detectMultiScale(roi, 1.1, 5)
```

```

38         for(ex,ey,ew,eh) in eyes:
39             cv2.rectangle(roi_c,(ex,ey),(ex+ew,ey+eh),(255,0,0),2)
40
41         smiles = smile_cascade.detectMultiScale(roi, 1.7, 20)
42         for(sx,sy,sw,sh) in smiles:
43             cv2.rectangle(roi_c,(sx,sy),(sx+sw,sy+sh),(0,0,255),2)
44
45         timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
46         cv2.imwrite(os.path.join(output_dir, f"capture_{timestamp}.jpg"
47             ), frame)
48
49         cv2.imshow("Detection", frame)
50         if cv2.waitKey(1) & 0xFF == ord('q'):
51             break
52
53     cap.release()
54     cv2.destroyAllWindows()

```

Chapitre 4

Jeu de données et captures

Les sept captures suivantes illustrent le fonctionnement du système.

4.1 Captures d'écran

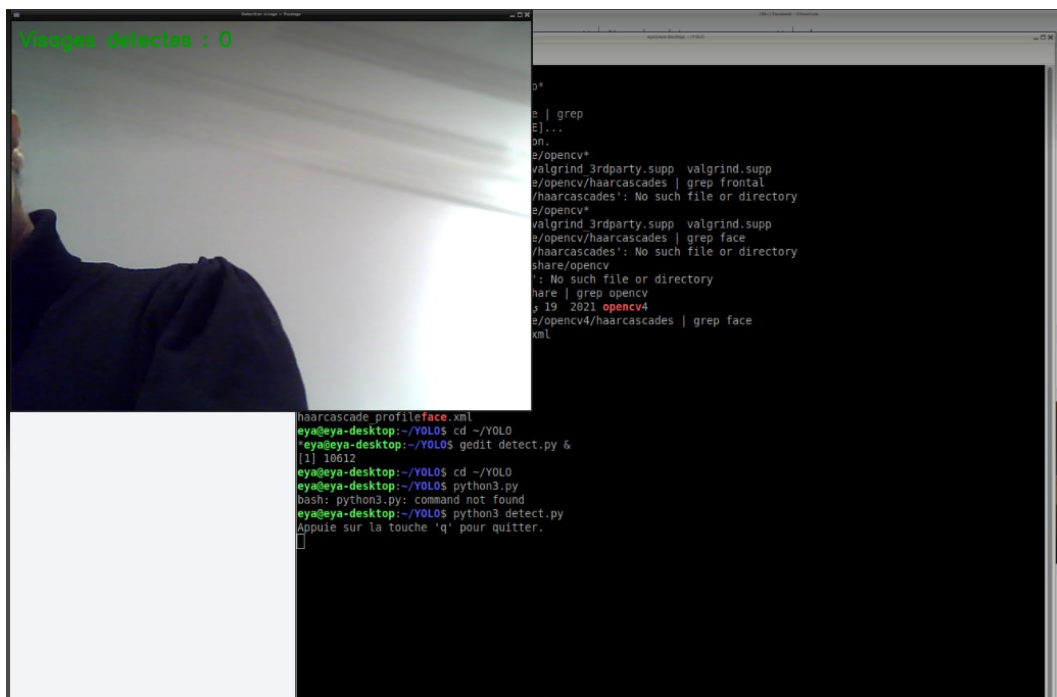


FIGURE 4.1 – Détection initiale du visage

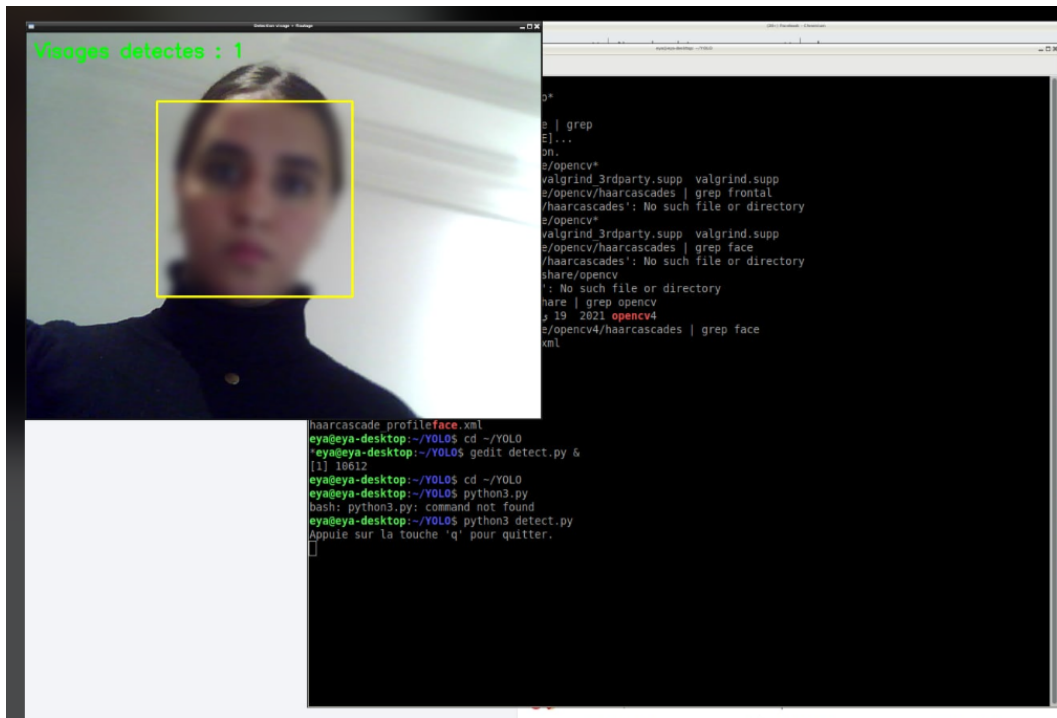


FIGURE 4.2 – Détection visage + annotations

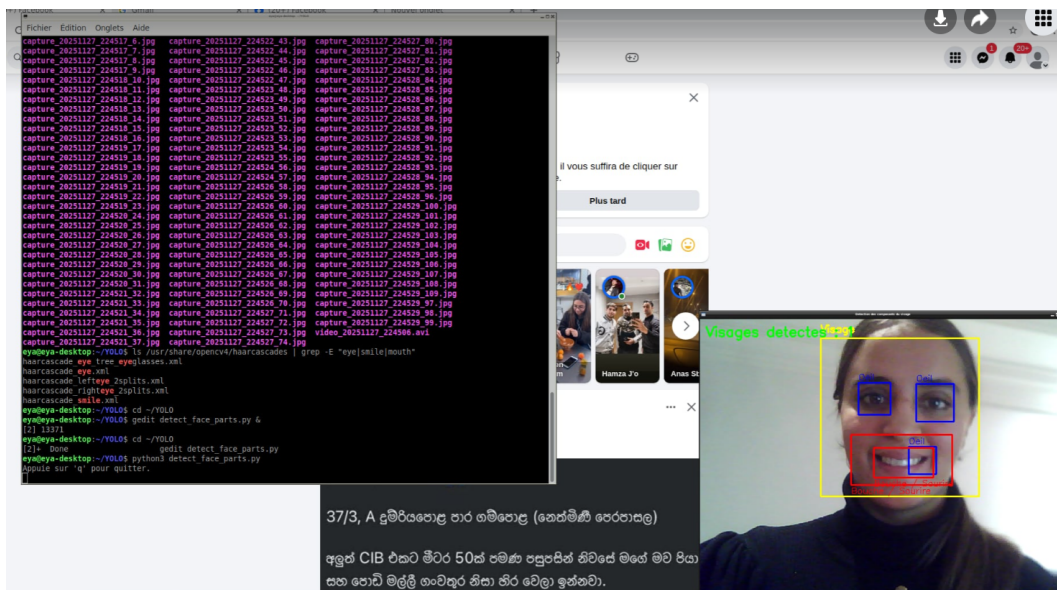


FIGURE 4.3 – Détection des yeux

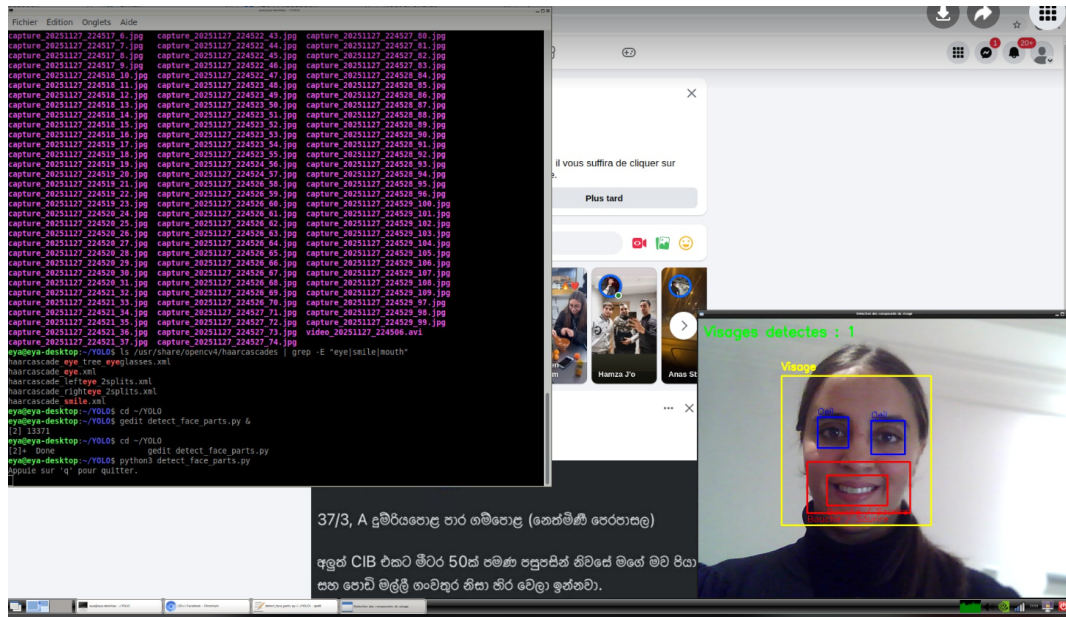


FIGURE 4.4 – Détection du sourire

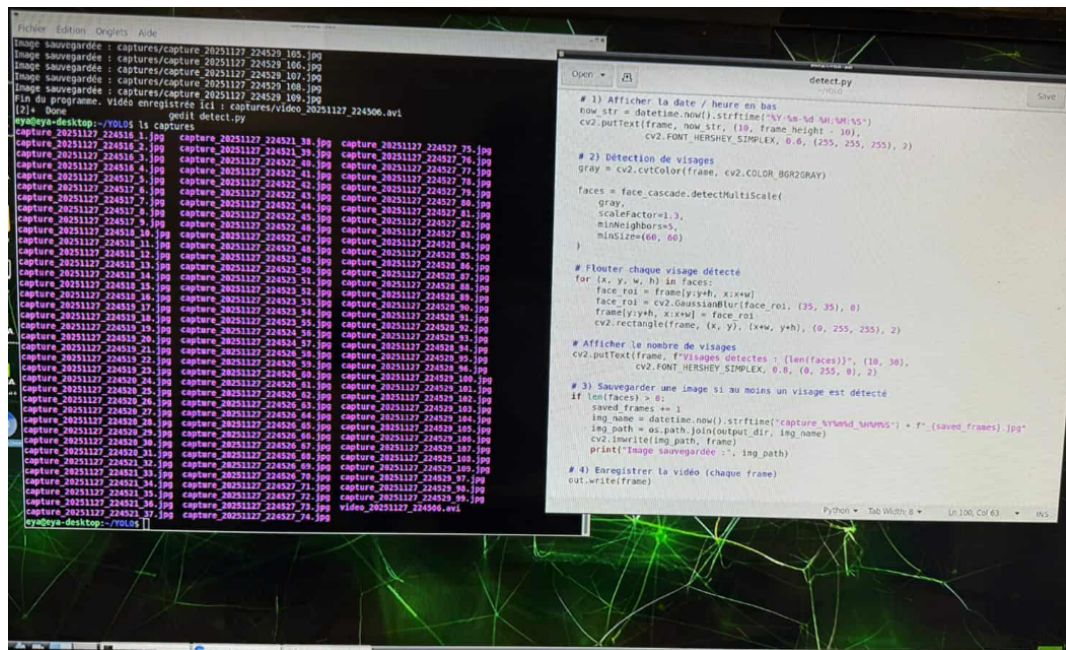


FIGURE 4.5 – Horodatage en temps réel

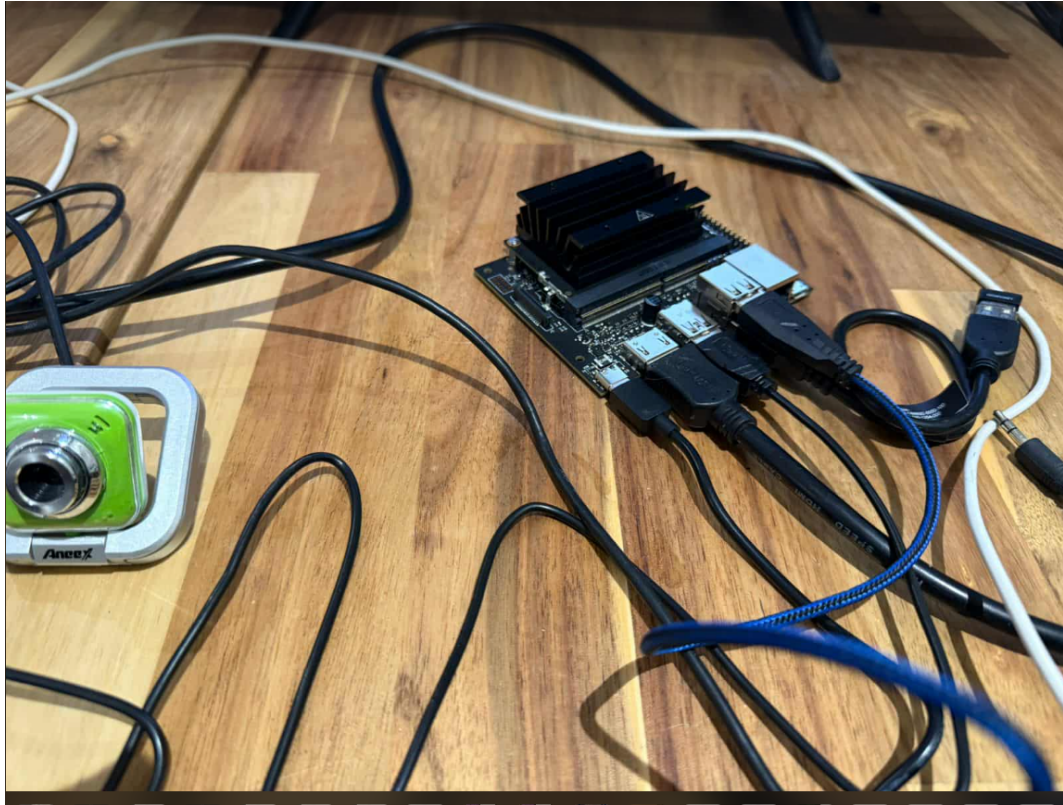


FIGURE 4.6 – Câblage et environnement (1)

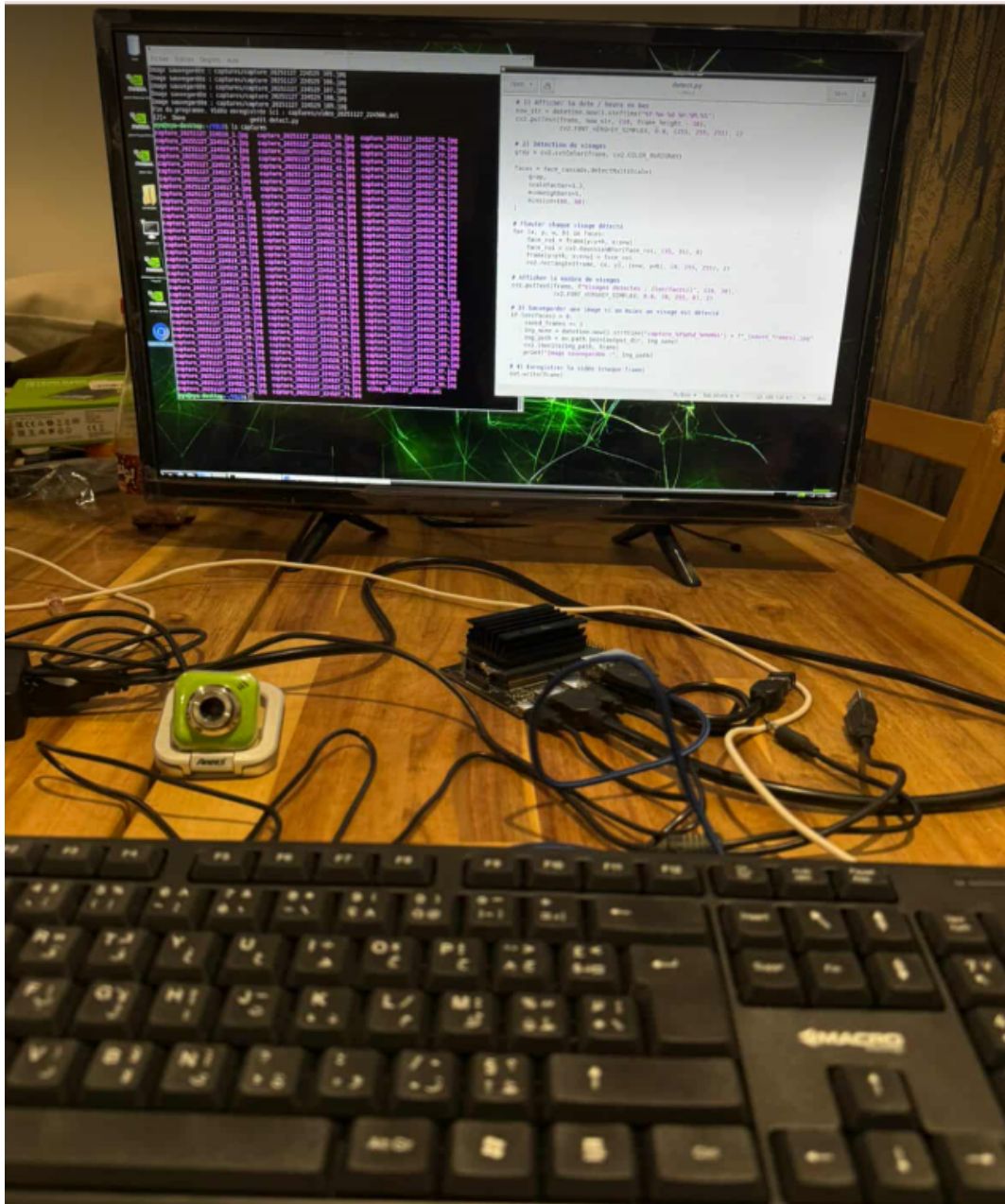


FIGURE 4.7 – Câblage et environnement (2)

Chapitre 5

Conclusion

Ce projet a permis d'implémenter un système complet de détection faciale en temps réel. Les outils et bibliothèques utilisés garantissent une exécution fluide et fiable. La carte NVIDIA offre une perspective d'amélioration pour des modèles IA plus avancés.